Seasonal Farm Market - Design Document

Hackathon: Build an RL Environment That Can't Be Broken | Team: prple | Demo: farm_seasons

1. OVERVIEW & SYSTEM ARCHITECTURE

Seasonal Farm Market is an unbreakable RL environment where an AI agent acts as a farm business manager across a multi-day horizon via 11 strictly typed domain tool calls. Built strictly using the Python standard library (clean Docker build in under 30s; zero third-party packages in sandbox runtime), the system enforces total isolation through a dual-container architecture:

- Sealed Sandbox Container: Hosts game state; isolated via internal: true (no internet access), read-only root filesystem, cap_drop ALL, and no-new-privileges. Exposes ONLY 11 domain tools - zero bash, code execution, or file read/write tools exist.
- Orchestration Harness: Connects to any OpenAI-compatible API, enforces 40-step and 10-minute live limits, caps tool output at 2KB, compresses conversational history (<64K tokens), and performs tamper-proof scoring by replaying the validated action log through an independent engine.

2. TASK DESIGN, PROCEDURAL VARIATION & DIFFICULTY CURVE (16 TASKS ACROSS 3 TIERS)

- Easy Tier (farm_basic, farm_two_crops, farm_sprint): Tests core plant-harvest-sell loop, yield vs. grow-time trade-offs, and capital velocity over 10-15 day horizons.
- Medium Tier (farm_seasons [DEMO], farm_events, farm_spoilage): Sinusoidal seasonal prices, shelf-life inventory decay, storage limits (60 units), and weather/demand shocks (50% drought yield, 1.6x price spikes).
- Hard Tier (farm_credit, farm_crash, farm_drought, farm_market_rush): Compounding commercial debt, high price slippage (1.5-2.0%/unit), noisy forecasts, and extreme price volatility.
- Microeconomic Benchmarks (econ_cobweb_cycle, econ_cournot_elasticity, econ_dutch_disease, econ_hotelling_arbitrage, econ_pigouvian_commons): Deep structural economic phenomena (production lag gluts, quadratic price impact, resource curse, exhaustible scarcity, tax internalities). Zero prompt leakage.
- Procedural Variation by Seed: Seeds vary base prices (+/-15%), seed costs (+/-10%), yields (+/-1), shock timing (drought onset/duration), and seasonal phases. The oracle dynamically recalculates target per seed; 1,600/1,600 automated gate checks pass with oracle >= 0.95 and no-op <= 0.05.

3. DETERMINISTIC PROGRAMMATIC REWARD FORMULATION

The verifier is 100% deterministic, programmatic, and computed strictly in integer cents (no float rounding exploits):

net_worth = cash + sum(inventory_units * min(floor_price, current_market_price)) - debt [floor_price = 50% of base_price]
gain = net_worth - start_cash | ratio = gain / oracle_gain | oracle_benchmark in [0.960, 0.985]
profit = oracle_benchmark * ratio (if ratio <= 1) else smooth non-saturating curve up to 1.0
total_score = profit * (0.85 + 0.15 * debt_ok) [debt_ok = 1 if debt == 0 at finish else 0]

Key Reward Integrity Mechanics:

- Anti-Hoarding Valuation: Unsold inventory is valued at min(floor_price, current_market_price). Hoarding during crashes creates zero phantom value.
- Independent Non-Saturating Scoring: Eliminates artificial 100% clamping so strong models are evaluated continuously without false ceilings.
- Daily Land Tax Drain: Land taxes apply daily ($15-$25/plot/day). Inaction (no-op) erodes capital and strictly guarantees score <= 0.05.
- Insolvent Debt Penalty: Unpaid debt carries compounding interest and incurs a 15% score haircut if non-zero at finish.

4. RED TEAM ATTACK MATRIX & SECURITY AUDITING

Empirical verification across 7 adversarial attacker vectors (scripts/red_team.py):

Attack Vector	What It Tests	What Happened (Observed)	Engine Defense Mechanism
Hoarder Agent	Unsold inventory gives unearned score	Scored 0.0000; phantom value wiped	Valued at min(floor, market); daily land tax erodes unliquidated stock.
Counterfeit Selling	Selling > owned or negative quantity	Rejected with error; cash unchanged	Strict bounds (qty in [1, owned]); rejects negative/zero sales before mut
Deadbeat Borrower	Take max loan and never repay	Scored 0.0000; total score crushed	Debt deducted from net worth; 0.85 debt_ok multiplier; compounding int
TimeWarp / Null Args	advance_days(9999), null, bad types	Clamped to 7 days; 0 server crashes	Strict type coercion & [1, 7] bounds validation; invalid actions safely reje
Plot Squatter / Fake Crop	Plant on occupied plot or fake crop	Rejected with error; plot preserved	Plot occupancy validation & crop catalog check; episode continues unha
Tool Flooder	Spam 100 tool calls in single reply	Calls 11-100 dropped; flagged	Hard cap MAX_TOOL_CALLS_PER_REPLY = 10; memory & executio
Garbage Speaker	Truncated JSON, raw conversational text	Parsed candidates; 0 crashes	Dual-mode parser with regex fallback & normalizer; resilient token extra

5. 10,000-STEP INVARIANT FUZZING & CRYPTOGRAPHIC REPLAY TOOL

- 10,000-Step Adversarial Invariant Fuzzing (scripts/fuzz_engine.py):
  * Automated fuzzer bombarded the engine with 10,000 random/adversarial action sequences across 104+ full episodes.
  * Checked 3 mandatory invariants every step: (1) Cash never silently goes negative without debt, (2) Inventory is never negative, (3) Identical seed + action sequence yields identical SHA-256 state hash.
  * Results: Exactly 0 unhandled crashes and 0 determinism divergences across all 10,000 checks.
- Cryptographic Replay Verification Tool (scripts/verify_replay.py):
  * Verifies log authenticity by reading raw episode.jsonl, resetting a fresh engine with the logged seed, and executing logged actions.
  * Compares state hashes at every turn: Verified 100% bit-for-bit parity on sample-trajectories/success (8/8 steps) and failure (9/9 steps).

6. MODEL CAPABILITY SEPARATION & MEASURED ECONOMIC BENCHMARKS

- Empirical Model Separation: Weak Model (Random/No-op: 0.0000) vs. Mid Model (Greedy/Myopic: 0.0219 - 0.3623) vs. Strong Model (Oracle: 1.0000). Separation spread ranges from +0.6377 on basic farming up to +1.0000 on deep microeconomic cycles.
- Measured Empirical Economics (scripts/verify_economics.py - Real Engine Runs):
  * Cournot Elasticity: Naive $73.27 (score 0.2361) vs. Strategic $171.04 (score 1.0000) [+0.7639 spread]
  * Cobweb Production Cycle: Naive $44.69 (score 0.0000) vs. Counter-Cyclical $282.72 (score 1.0000) [+1.0000 spread]
  * Liquidity Mismatch: Naive -$15.54 (score 0.0000) vs. Solvent Staggering $143.76 (score 1.0000) [+1.0000 spread]
  * Hotelling Arbitrage: Naive $39.15 (score 0.0000) vs. Scarcity Arbitrage $159.43 (score 1.0000) [+1.0000 spread]
  * Dutch Disease: Naive $23.40 (score 0.0000) vs. Diversification $194.68 (score 1.0000) [+1.0000 spread]

7. CONTEXT MANAGEMENT, LIVE LIMITS & SECURITY TELEMETRY

- Context Protection: Tool output hard-capped at 2,000 characters; older observations compressed into 1-line state digests while preserving the last 3 turns intact. 40-step episodes consume <18K tokens, staying safely below 64K limits.
- Live Limits: 60-day horizon finishable well within 40 steps via advance_days(1-7) and multi-tool batching (up to 10 calls/reply hinted in schemas).
- Anomaly Detection: Harness logs suspicious_count and invalid_actions_count, emitting [SECURITY WARNING] in terminal and summary.json.